UNITÀ 4 · APPLICAZIONI PER SISTEMI MOBILI · LEZIONE 5

# La modifica del layout: TextView e ImageView

QuizTPSI prende forma: un titolo, una domanda, un'immagine. Impariamo a disporre le View in XML, a usare le risorse e a cambiarle dal codice Kotlin.

**5G TPSI** Layout, risorse, TextView e ImageView

PER COMINCIARE

# Che cosa ti fa disinstallare un'app più in fretta?

SONDAGGIO

- **A** Un'interfaccia confusa
- **B** Troppa pubblicità
- **C** È lenta o si blocca
- **D** Chiede troppi permessi
- **E** Testi piccoli o illeggibili

COME È FATTA UNA SCHERMATA

# Un albero di View

- LinearLayout (verticale) · id main
  - TextView · tvTitolo
  - ImageView · ivDomanda
  - TextView · tvDomanda
  - Button · lezione 6

- Una **View** disegna qualcosa: testo, immagine, pulsante.
- Un **ViewGroup** contiene altre View e decide dove metterle.

> **Il design non è solo come una cosa appare e come la senti. Il design è come funziona.**
>
> **Steve Jobs**, intervista al New York Times Magazine, 2003

I CONTENITORI

# Tre ViewGroup che incontrerai subito

### LinearLayout

Mette i figli in fila, in verticale o in orizzontale. Semplice: lo usiamo oggi.

### ConstraintLayout

Ogni View è "legata" ai bordi o alle altre con vincoli. È il contenitore del modello di progetto.

### FrameLayout

Sovrappone i figli uno sull'altro: utile per un'immagine con un testo sopra.

| Attributo | Che cosa fa |
|---|---|
| `match_parent` | Grande quanto il contenitore |
| `wrap_content` | Grande quanto il contenuto |
| `padding / layout_margin` | Spazio dentro il bordo / spazio fuori dal bordo |

METTITI ALLA PROVA

# Un TextView ha uno sfondo arancione. Aumenti `android:padding`: che cosa succede?

QUIZ

A Il TextView si allontana dagli altri, lo sfondo resta uguale

B Il testo diventa più grande

C Il testo si allontana dal bordo e l'area arancione si allarga

D Niente, se la larghezza è wrap_content

IL LAYOUT IN XML

# activity_main.xml con un LinearLayout

```
<LinearLayout xmlns:android="…/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvTitolo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="QuizTPSI"
        android:textSize="28sp"
        android:textStyle="bold" />
</LinearLayout>
```

- `@+id/…` crea un id nuovo nella classe R.
- L'id `main` serve al codice generato in MainActivity: **non toglierlo**.
- Testi in **sp**, distanze in **dp**.
- Il secondo TextView, `tvDomanda`, è uguale con 20sp e `layout_marginTop="16dp"`.

AL COMPUTER · 20 minuti

# La schermata della domanda

1. Apri **activity_main.xml** in vista **Code** o **Split**.
2. Sostituisci il ConstraintLayout con un **LinearLayout** verticale, mantenendo `android:id="@+id/main"`.
3. Togli il TextView "Hello World!" e aggiungi **tvTitolo** e **tvDomanda** come nella slide.
4. In tvDomanda scrivi: *Android usa un kernel Linux*.
5. Prova un `android:background` e un padding su tvDomanda.

**Controllo:** l'anteprima mostra il titolo in grassetto in alto e la domanda sotto. Se compare *Unresolved reference: main*, hai tolto l'id dal LinearLayout.

LE RISORSE

# Niente testi scritti nel layout

```
<!-- res/values/strings.xml -->
<resources>
  <string name="app_name">QuizTPSI</string>
  <string name="titolo">QuizTPSI · Android</string>
  <string name="domanda_1">Android usa un
      kernel Linux.</string>
  <string name="contatore">Domanda %1$d
      di %2$d</string>
</resources>

<!-- nel layout -->
android:text="@string/titolo"
```

- Tradurre l'app: si aggiunge `values-en/strings.xml`, il codice non cambia.
- Colori in `colors.xml`, stili in `themes.xml`.
- Nel codice: `getString(R.string.titolo)`.
- Attenzione all'apostrofo: in strings.xml si scrive `L\'app`.

AL COMPUTER · 15 minuti

# Sposta i testi nelle risorse

1. In **strings.xml** aggiungi `titolo`, `domanda_1` e `contatore` come nella slide.
2. Nel layout usa `@string/titolo` e `@string/domanda_1`.
3. In **colors.xml** crea `arancio` = `#D0541F` e usalo come colore del titolo (`android:textColor`).
4. Aggiungi una stringa con un apostrofo, per esempio *L'app è pronta*, e fai Build.

**Controllo:** gli avvisi gialli *Hardcoded string* spariscono. Con l'apostrofo senza `\'` la build fallisce: correggilo e riprova.

LE IMMAGINI

# ImageView e la cartella drawable

```
<ImageView
    android:id="@+id/ivDomanda"
    android:layout_width="match_parent"
    android:layout_height="200dp"
    android:layout_marginTop="16dp"
    android:scaleType="fitCenter"
    android:src="@drawable/img_domanda1"
    android:contentDescription=
        "@string/desc_domanda1" />
```

- Formati: PNG, JPG, WebP, oppure **vettoriali** XML, nitidi a ogni densità.
- `fitCenter` mostra tutta l'immagine; `centerCrop` riempie e ritaglia.
- `contentDescription` è il testo che **TalkBack** legge a chi non vede.

QUIZ A TRABOCCHETTO

# Quale nome di file è valido nella cartella `res/drawable`?

QUIZ

A logo_scuola.png

B Logo-Scuola.png

C logo scuola.png

D 2logo.png

AL COMPUTER · 15 minuti

# Aggiungi l'immagine della domanda

1. Tasto destro su **res/drawable**, **New**, **Vector Asset**: scegli un'icona da Clip Art e chiamala **img_domanda1**.
2. In alternativa copia un tuo PNG in drawable, con un nome valido.
3. Aggiungi l'ImageView tra tvTitolo e tvDomanda, come nella slide.
4. Crea la stringa `desc_domanda1` con una descrizione vera dell'immagine.
5. Esegui sul dispositivo.

**Controllo:** l'immagine compare sotto il titolo, alta 200 dp. L'avviso giallo sulla `contentDescription` mancante non c'è più.

DAL CODICE

# Cambiare testi e immagini in Kotlin

### FINDVIEWBYID

```
val tvDomanda =
    findViewById<TextView>(R.id.tvDomanda)
val ivDomanda =
    findViewById<ImageView>(R.id.ivDomanda)

tvDomanda.text = getString(R.string.domanda_1)
ivDomanda.setImageResource(R.drawable.img_domanda1)
```

### VIEW BINDING

```
// build.gradle.kts (:app)
android { buildFeatures { viewBinding = true } }

// MainActivity
val b = ActivityMainBinding.inflate(layoutInflater)
setContentView(b.root)
b.tvDomanda.text = getString(R.string.domanda_1)
```

> Va fatto **dopo** `setContentView`. Nei vecchi tutorial troverai `kotlin-android-extensions`: è stato ritirato, non usarlo.

AL COMPUTER · 15 minuti

# Il contatore delle domande

1. Aggiungi in cima al layout un TextView **tvContatore**, 16sp.
2. In `onCreate`, dopo setContentView, trova le tre View con `findViewById`.
3. Scrivi in tvContatore `getString(R.string.contatore, 1, 3)`.
4. Imposta da codice testo e immagine della domanda 1.
5. Esegui, poi prova a chiamare findViewById prima di setContentView e guarda Logcat.

**Controllo:** in alto compare **Domanda 1 di 3**. Con l'ordine sbagliato l'app si chiude e Logcat mostra una *NullPointerException*: rimetti l'ordine giusto.

PUZZLE

# Abbina ogni termine al suo significato

| | | |
|---|---|---|
| **wrap_content** | --- | Testo preso da strings.xml |
| **match_parent** | --- | Riempie il contenitore |
| **dp** | --- | Grande quanto ciò che contiene |
| **sp** | --- | Come dp, ma segue la dimensione del testo scelta dall'utente |
| **@string/titolo** | --- | Misura indipendente dalla densità dello schermo |
| **contentDescription** | --- | Descrizione letta da TalkBack |

LE INTERFACCE OGGI

# Lo stesso titolo, in un altro modo





BIGLIETTO D'USCITA

# Completa prima di uscire

Lo spazio interno, coperto dallo sfondo, è il _____; quello esterno è il margin.

Per le dimensioni dei caratteri si usa l'unità ____.

I testi dell'app stanno nel file _____.

La descrizione di un'immagine per TalkBack è l'attributo _____.

Una View si trova dal codice con _____, dopo setContentView.

⏱ 3:00

RIEPILOGO

# Cosa portiamo a casa

- Una schermata è un albero di View e ViewGroup
- LinearLayout verticale, id main da non togliere
- padding dentro, margin fuori; dp e sp
- Testi, colori e immagini nelle risorse
- ImageView sempre con contentDescription
- findViewById o View Binding, dopo setContentView

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.